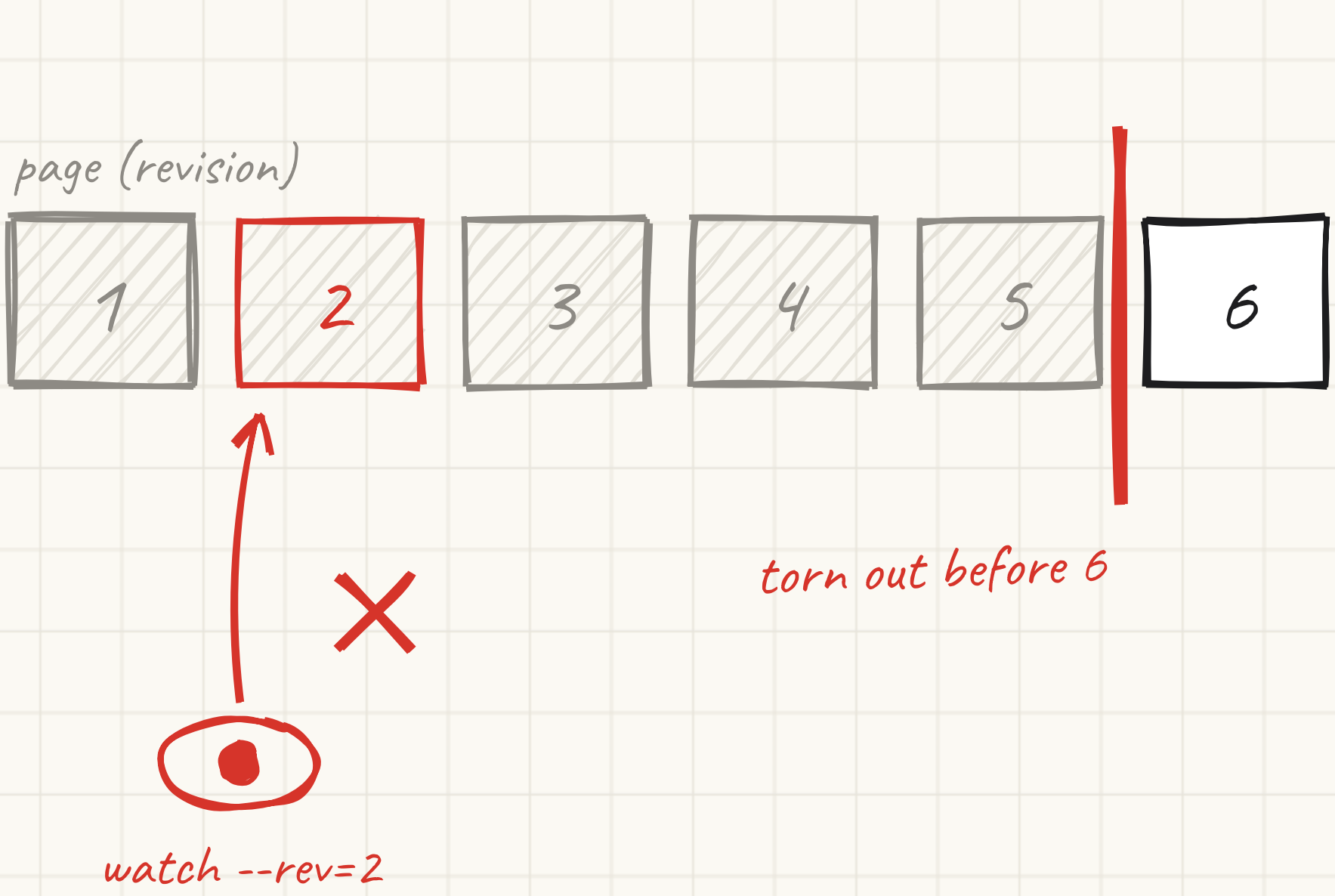
Your watch died. The page is gone.

etcd is a shared notebook. Old pages get torn out on purpose.

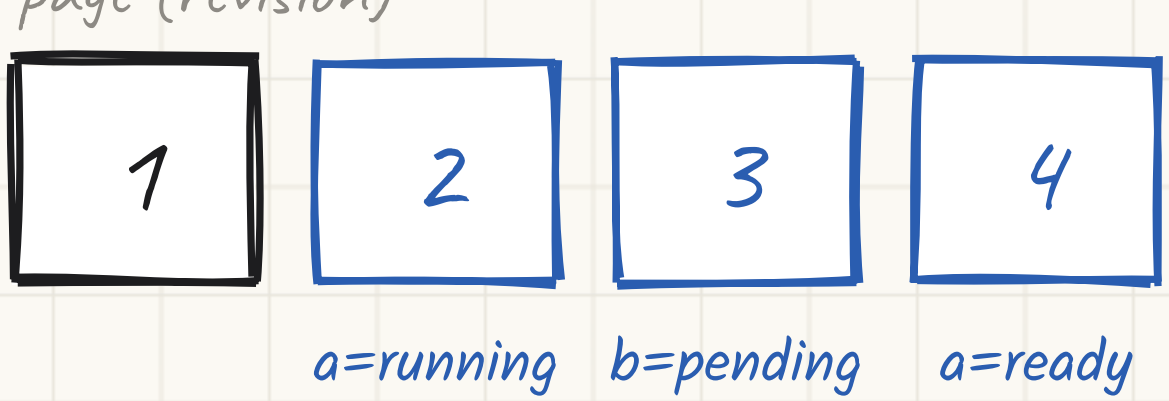


Apoorv Garg

swipe ->

Every change gets a numbered page.

The page number is the revision. Each change adds one page.

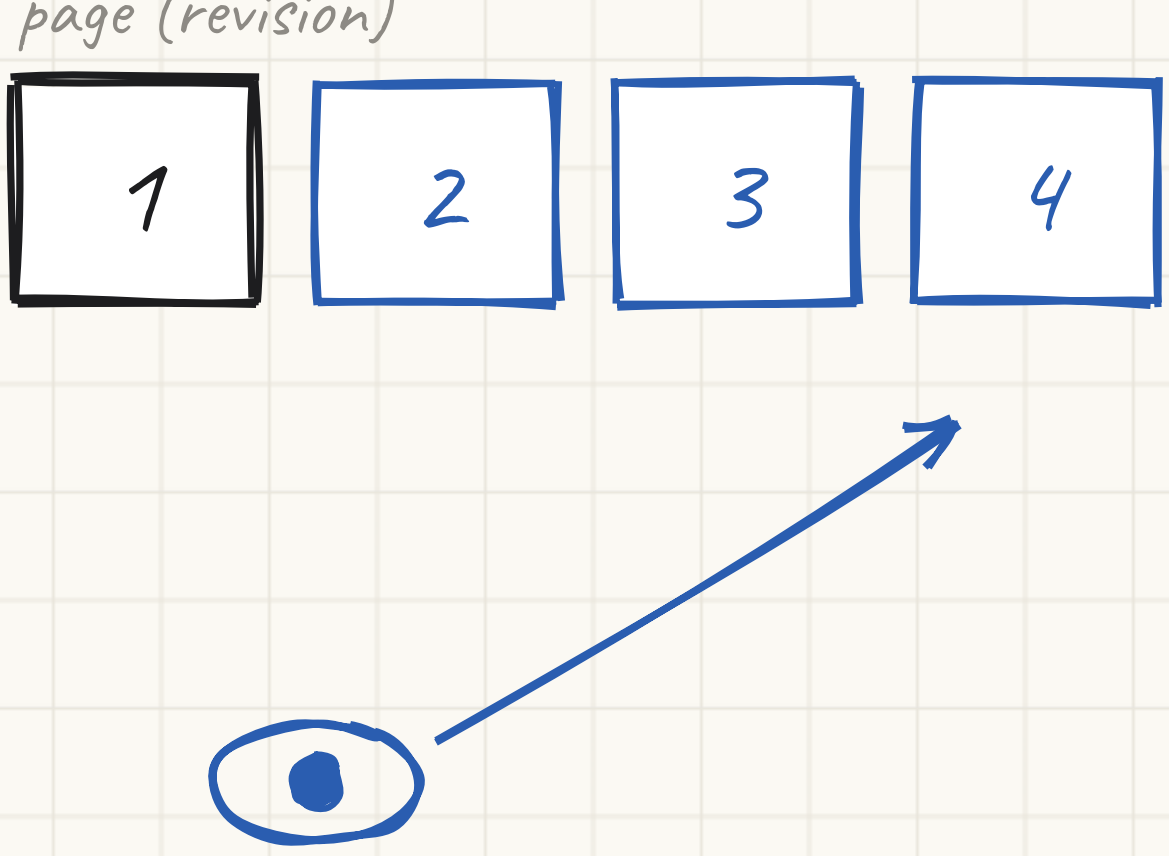


put a, put b, update a -> revision 4

So you can rewind.

Ask for everything since page 2.

That request is a watch. A controller that looked away gets the missed pages back.

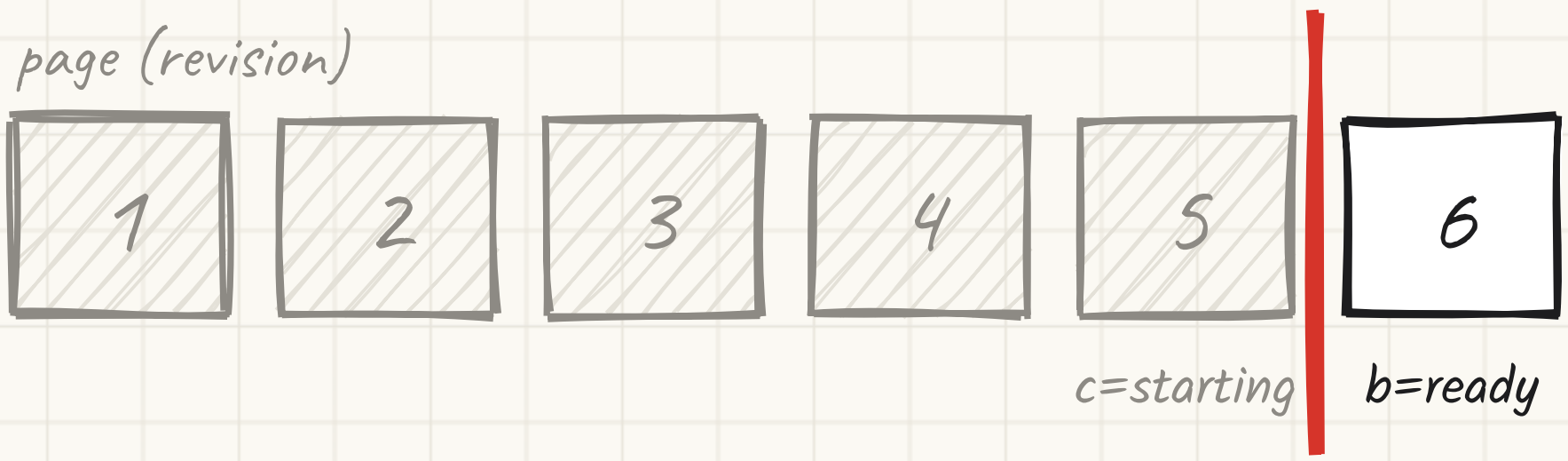


`events_seen=3` ✓

But history costs disk.

Old pages get torn out.

That is compaction. Two more changes, then `compact 6`. Old versions before 6 are gone.

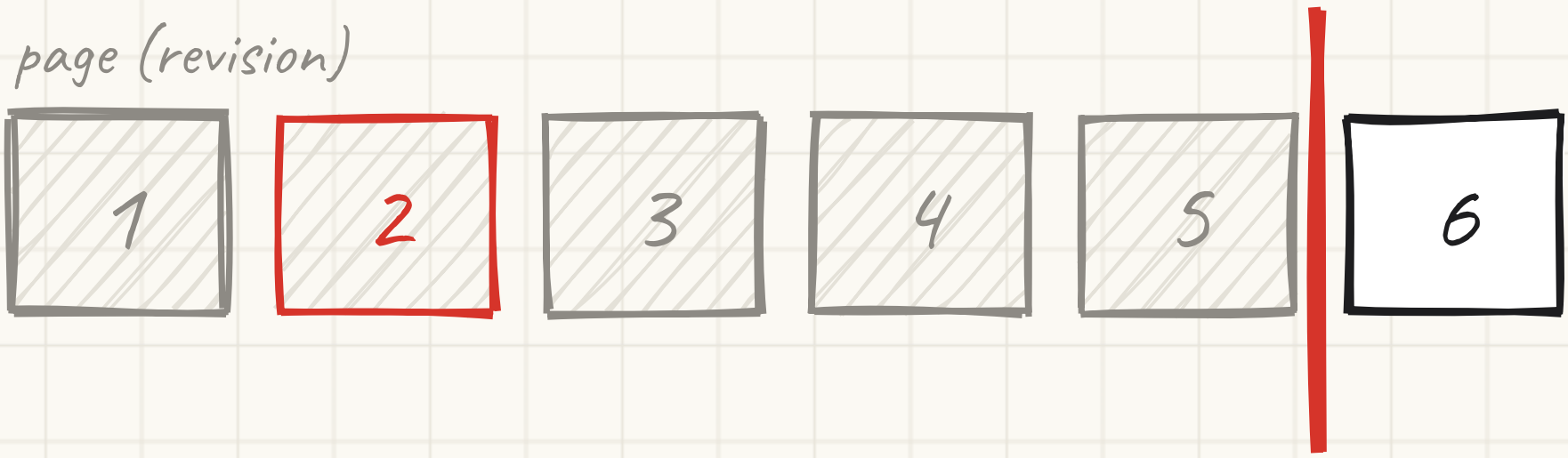



```
compacted revision 6
```

Now your controller reconnects.

Wrong: ask for page 2 again.

etcd says that page is gone. Real output:



torn out before 6



watch --rev=2

```
watch was canceled (etcdserver: mvcc:
required revision has been compacted)
```

Retry it? Same answer.

Asking again never helps.

Revision 2 will never come back. Even `get --rev=2` fails. Go calls it ErrCompacted.



a history gap, not a network blip

So what is right?

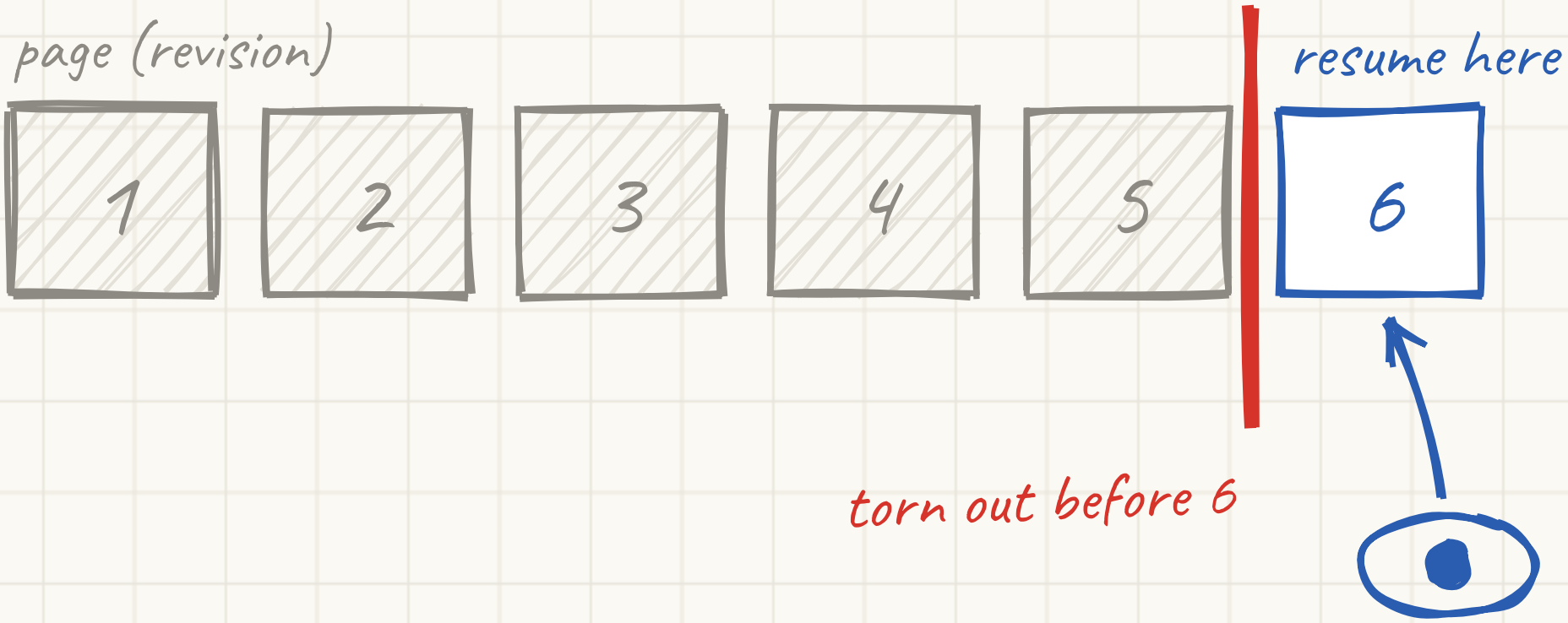
Right: read, then follow along.

Read the notebook as it is now. Then watch from a page that still exists.

1. `get --prefix /demo/pod/`

   ```
   a=ready b=ready c=starting
   ```

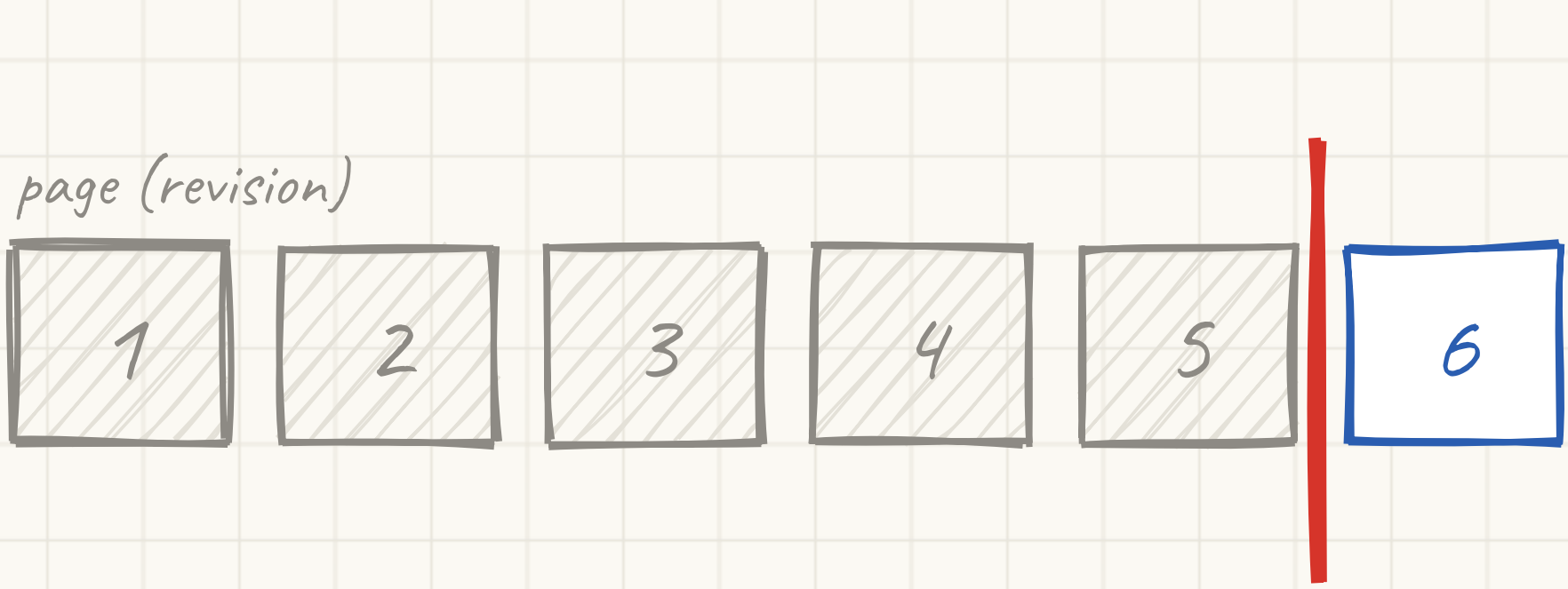
2. `watch --rev=6`



One catch.

Torn pages are not replayed.

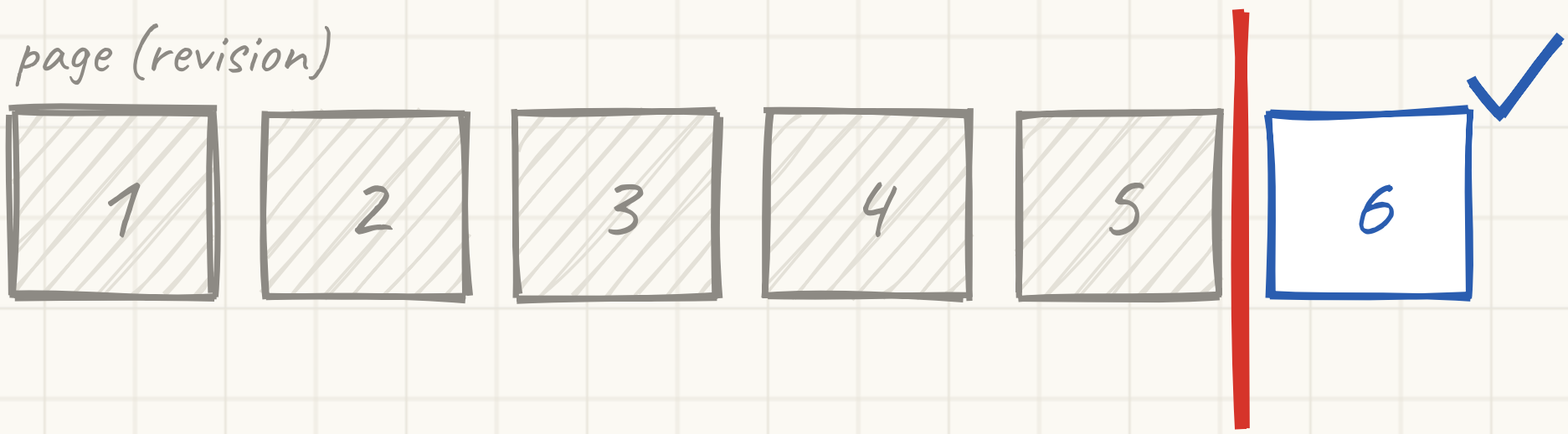
Changes on torn pages never come back as events. What you read is the truth now.



So, the whole drawing.

So, the whole drawing.

Pages get numbers. Old pages get torn out. Page gone? Read the notebook, then follow along.



How does your controller handle ErrCompacted today?